ShortCircuit: AlphaZero-Driven Generative Circuit Design

Dimitris Tsaras[†‡], Antoine Grosnit[‡], Lei Chen[‡*], Zhiyao Xie[†], Haitham Bou Ammar[‡], Mingxuan Yuan[‡]
Hong Kong University of Science and Technology[†], Noah's Ark Lab Huawei[‡]
Hong Kong, London

Abstract

Traditional logic synthesis heuristics have plateaued. We introduce ShortCircuit, a novel transformer-based architecture generating AND-Inverter Graphs (AIGs) directly from truth tables. Unlike prior deep learning efforts, we employ a two-phase training process combining supervised learning and an AlphaZero variant to navigate the doubly exponential state space. Evaluated on 500 8-input truth tables from real-world circuits, ShortCircuit guarantees functional correctness and outperforms the state-of-the-art tool ABC in circuit size. Furthermore, our greedy rollout mechanism achieves a 31× speedup, demonstrating significant efficiency gains.

1 Introduction

The rapid rise of AI has driven computational demands beyond current hardware capabilities, creating a major bottleneck. Chip design is key to advancing next-generation computing, but traditional methods struggle to keep pace, highlighting the need for faster and more innovative approaches. At its core, a chip is the physical embodiment of a Boolean function, transforming binary inputs into desired outputs. Creating these embodiments is facilitated by logic synthesis, a crucial step in chip design that converts functional descriptions into graphs connecting logic gates. The logic synthesis process must balance power, performance, and area, posing a complex optimization challenge. In this paper, we investigate the use of Machine Learning (ML) to generate optimized digital circuits directly from Boolean logic specifications, offering a fresh perspective on the chip design process.

Truth tables fully specify Boolean functions by listing outputs for all input combinations. Consequently, we use truth tables as the input Boolean logic description for our problem. Our method outputs directed acyclic graphs (DAGs) as AND-Inverter Graphs (AIGs), a standard logic structure in Electronic Design Automation (EDA) [24, 39]. AIGs only use 2-input AND gates and inverted or normal edges, offering a simple, scalable, and widely adopted representation that makes them an ideal choice for our ML-based circuit generation approach.

Traditional logic optimization methods optimize large AIGs by forming small cuts, finding smaller equivalent representations, and replacing the original subgraphs. The most effective method, `rewrite`, forms 4-input cuts and matches their truth tables to a precomputed database for replacements [25]. While industrial approaches have explored scaling to 8-input cuts, the double exponential growth of the truth table space limits the coverage of such databases [3]. Moreover, the structural rigidity of AIGs and the complexity of the problem result in most cuts failing optimization, limiting the effectiveness of these methods [36]. Recent works aim to speed up chip design by applying ML across EDA stages [12, 16], including placement [38], routing [1], and logic synthesis [37]. Rather than directly tackling the graph generation problem, most ML methods for logic synthesis focus on the optimization of synthesis recipes, which are sequences of operators acting on a logic graph to modify its structure while preserving the associated Boolean function. More recently, deep-generative methods emerged aiming to generate logic graphs rather than operator sequences [7, 9, 21]. The generative approach offers higher potential as it offers more flexibility than working with a fixed set of operators; however, this requires exploring a larger space, due to the double exponential growth of the search space with the number of Boolean inputs.

Such a vast search space renders traditional ML methods ineffective for optimal AIG generation. However, recent advances have demonstrated that tailored model architectures and exploration-exploitation-aware training protocols can achieve remarkable performance even in tasks involving such large spaces. Indeed, the success of methods like AlphaGO [34], AlphaZero [35], AlphaFold [17] and even the emergence of large language models [8, 18] relied on the development of custom model architectures capturing structural properties of board games, proteins, or language. Moreover, the training of these models either leverages naturally abundant data or employs specific data-augmentation and exploration-exploitation strategies to improve their performance.

We propose ShortCircuit, a new transformer-based architecture structurally adapted for generating AIGs. Our transformer takes logic nodes represented as truth tables as input, and each forward pass predicts the next AND node to create in order to realize a target truth table. We further utilize an AlphaZero policy to navigate the large state space and discover more compact designs. We make the following contributions: **i)** we formally define the challenging problem of generating AIGs from target truth tables, characterized by a doubly exponential state space and a quadratically growing action space. **ii)** We introduce ShortCircuit, a novel AIG-aware architecture, enabling effective exploration of this vast search space that guarantees the correctness of the produced AIG. **iii)** We develop a two-stage training scheme combining supervised learning and reinforcement learning to improve generalization and scalability. Finally, **iv)** we demonstrate the effectiveness of ShortCircuit by producing circuits 18.62% smaller compared to the state-of-the-art logic synthesis tool ABC [26], while achieving a 31× speedup in its greedy variant, showcasing the potential of ML methods to revitalize the field of logic synthesis with a fresh perspective.

We present background and related work in Section 2, followed by the problem formulation in Section 3. We then describe our model architecture in Section 4 and our tailored training procedure in Section 5. We finally provide the empirical evaluation of ShortCircuit in Section 6.

2 Background

A digital circuit cascades logic gates to realize a Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, mapping n-bit inputs to m-bit outputs. An And-Inverter Graph (AIG) is a Directed Acyclic Graph (DAG) that is commonly used to represent such functions at the early stage of the chip design process.

*Corresponding author: lc.leichen@huawei.com

2.1 AND-Inverter Graphs

An AIG is composed of three types of nodes: (1) primary inputs, which we also refer to as inputs, (2) primary outputs, which we call the outputs, and (3) 2-input AND-nodes representing the logic gate AND. In this work, we focus on the generation of single-output AIGs that represent Boolean functions of the form $f : \{0,1\}^n \rightarrow \{0,1\}$ as they play an important role in logic synthesis. Fig. 1 illustrates the structure of an AIG with $n = 3$ inputs, where $\{I_k\}_{1 \leq k \leq 3}$ represent input nodes, $\wedge_4, \wedge_5$ are AND gates, and O is the output. Edge orientation indicates the direction of the Boolean signal propagation from one node (called fanin) to another (called fanout). Moreover, the two types of edges, plain and dashed, in Fig. 1 indicate that the Boolean signal can be inverted when going from a fanin to a fanout. The primary output is always connected to a single AND node with a direct or an inverter link. As AIGs only contain AND operations and Boolean inversions, we can map them to a canonical form (CNF). For instance, the CNF, naturally derived from its topology, of the AIG in Fig. 1 is $O = \neg(\neg(I_1 \wedge I_2) \wedge I_3)$, and we can conversely easily go from a CNF to an AIG. Moreover, applying equivalence-preserving operations to the CNF produces new CNFs that still encode the same Boolean function. Similarly, topologically distinct AIGs can realize the same function, and to compare the quality of two AIGs, a primary criterion is to compare their sizes, measured by the number of gates they contain [24]. Smaller AIGs are generally preferred as they simplify subsequent tasks such as placement and routing, and lead to more efficient circuits.

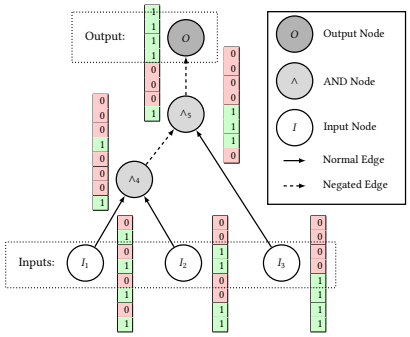


Figure 1: Representation of an AIG, showing the truth table associated to each node.

I_3	I_2	I_1	I_1	I_2	I_3	$\wedge_4$	$\wedge_5$	O
0	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	0	1
0	1	0	0	1	0	0	0	1
0	1	1	1	1	0	1	0	1
1	0	0	0	0	1	0	1	0
1	0	1	1	0	1	0	1	0
1	1	0	0	1	1	0	1	0
1	1	1	1	1	1	1	0	1

Table 1: Truth table of each node in the AIG from Fig. 1

2.2 Truth Tables

As for any other logical graphs, we can capture the behavior of an AIG by propagating Boolean values from its primary inputs to its primary outputs and applying the logical operations encountered on the directed paths. Considering again the exemplar AIG from Fig. 1, if $I_1 = 1$, $I_2 = 1$, and $I_3 = 0$, propagating the Boolean signals we can verify that the AIG output at O is 1. By enumerating all possible input combinations and recording the corresponding output values for each gate, we can build the AIG's full truth table, as shown in Table 1. Each row corresponds to the values of the AIG nodes for a specific set of entries $(I_1, I_2, I_3) = (i_1, i_2, i_3) \in \{0,1\}^3$ displayed on the left part of the table. We can extract from this representation a binary vector of size 2^n for each AIG node. For instance, the "truth table" vector representation of node $\wedge_4$ is $(0\,0\,0\,1\,0\,0\,0\,1)^\top$, and the primary output one is $(1\,1\,1\,1\,0\,0\,0\,1)^\top$. After discussing related works in more detail, we explain in the next section how our method leverages this rich vector representation for AIG generation.

2.3 Related works

Heuristics for AIG Generation and Optimization. As the inference of CNFs using exact SAT solvers often lead to exponentially large expressions, various heuristics such as Karnaugh maps [19], or Quine-McCluskey methods [22, 29, 30]), and algorithms [33] have been designed to obtain more compact expressions or circuits. Further efforts accompanying the rise in chip demand led to the development of widely used logic synthesis libraries that implement equivalence-preserving Boolean network operators. The open-source library `ABC` [26]) notably comprises dozens of logic graph operators aiming at reducing a network size or depth [23, 24]. Interestingly, some important operators such as `resub` or `rewrite` [6, 25] acts on the subject graph through a series of local modifications involving small single-output AIGs. Besides, applying a single operator on a logic network is suboptimal compared to applying several operators sequentially, though finding the best sequence is also a hard problem [31].

Machine Learning for Logic Synthesis. Many ML approaches have been explored to tackle the operator flow optimization progress. Some stateless optimization methods, such as Bayesian optimization [10, 11], search for the best flow without considering the subject graph specificities. Alternatively, state-based methods formulate the operator sequence optimization as a Reinforcement learning problem, and train policies on selected features of the logic network. While some works use high-level statistics of the subject graph (e.g., its number of nodes) [15, 28, 43], others rely on tailored graph convolutional networks (GCN) to extract richer features at the cost of a longer training time [4, 13, 27, 44]. Similarly, standard deep network architectures, such as CNNs [41], LSTMs [42], or GCNs [40] have been trained to predict the quality of a logic synthesis flow in a supervised way. Contrary to these works, we target AIG generation itself and not operator optimization.

Learning to Generate One Circuit at a Time. A first approach to generate Boolean networks with deep neural networks consists of substituting the gates and wires of a logic circuit by learnable nodes and connections to form a neural network [5, 14, 45]. The network parameters are learnt by minimizing the error made by forward passes compared to the target binary output. On the one hand, this method allows to cope with larger number of primary inputs as, instead of learning an entire family of logic graphs (e.g., the 8-input AIGs), it is specialized on one particular target truth table. On the other hand, it requires to train a new neural network for each target truth table, representing a significant runtime bottleneck. Therefore, several works inspired by the development of foundational generative models have followed another direction, consisting in learning the synthesis process itself with a deep neural network.

Learning Circuit Synthesis using Deep Learning. While [32] use a CNN backbone to generate prefix circuits by adding or deleting nodes in a $N \times N$ grid, [20] employ an auto-regressive diffusion model to generate DAG for high-level synthesis stage , and [9] design a two-level GNN architecture to synthesize analog circuits that work with non-binary signals. Closer to our work are Boolformer [7] and Circuit Transformer (CT) [21], both tackling digital network synthesis with an auto-regressive transformer-based architecture. They train their policies with a supervised training phase, predicting the next element of a logic graph given a target truth table, and doing inference through beam-search or MCTS simulations. Contrary to our work, Boolformer and CT use a symbolic representation of the Boolean formula associated to a logic graph that is encoded via depth-first search. Therefore instead of representing already built nodes by their truth tables and predicting the next node to add to the graph, they tokenize the symbols and let their model generate the next symbol of the logic formula (a logical operation, an input or an output), which requires more forward passes than for our method to produce a similar AIG. Besides, given a target truth table $T_\star$, Boolformer directly takes as input the boolean representation of $T_\star$ as we do, while [21] pass a (non-optimal) AIG realizing $T_\star$ to the CT to generate an improved logic network.

3 Problem Definition

Truth tables encompass the results for all possible input values, but do not provide sufficient structural information to derive an AIG representing that mapping. Heuristics that generate a Boolean expression, or an AIG, from a truth table lead to exponentially large solutions needing further refinement. Consequently, solving this AIG generation problem would significantly impact circuit design.

Problem Definition: Given a target truth table $T_\star \in \{0,1\}^{2^n}$, construct an AIG with the minimum number of nodes, such that its output node O has a truth table T_O matching $T_\star$.

Note that the size of a truth table associated with an n-input AIG is 2^n, thus, the set containing all truth tables of size 2^n has a cardinality of 2^{2^n}, which is on the order of 10^{77} for the space of truth tables that an 8-input AIG can represent. Exploring such a large space efficiently requires developing specific models and training techniques. These must exploit the structural properties of the problem while managing the exploration-exploitation trade-off inherent in such scenarios.

3.1 State Representation & Notations

We formally define an AIG with n inputs as a graph $\mathcal{G} = (V, E)$, where V and E represent the node and edge sets. To capture the dynamic nature of AIG construction, we introduce a temporal parameter, t, which simultaneously represents the current time step and the number of AND-nodes in the graph, denoted as $\mathcal{G}_t = (V_t, E_t)$. This allows us to model the evolution of the AIG over time, with the graph growing as new AND-nodes are added. We assign a unique integer ID to each node in the graph, regardless of its type; thus, at time t the node set is $V_t = \{I_1, \ldots, I_n, \wedge_{n+1}, \ldots, \wedge_{n+t}\}$, or with a node-type agnostic notation, $V_t = \{v_1, \ldots, v_{n+t}\}$. Following this notation, the graph $\mathcal{G}_0$ represents an AIG containing only input nodes, i.e., with $V_0 = \{I_1, I_2, ..., I_n\}$.

In our generative process, we encode the state corresponding to AIG $\mathcal{G}_t$ as a 3-tuple $s_t = (\mathcal{T}_t, T_\star, \mathcal{A}_t)$. Here, $\mathcal{T}_t = \{T_1, T_2, ..., T_{n+t}\}$ is the set of truth tables associated with the current nodes, $T_\star$ is the target truth table, and $\mathcal{A}_t = \{a_{n+1}, a_{n+2}, ..., a_{n+t}\}$ is the ordered set of actions performed so far. Each action generates a new AND-node by connecting two existing nodes in one of four possible configurations: (v_i, v_j), $(v_i, \neg v_j)$, $(\neg v_i, v_j)$, and $(\neg v_i, \neg v_j)$. Our goal is to perform a series of N actions, transforming $\mathcal{G}_0$ into a terminal AIG $\mathcal{G}_N$ such that the truth table of the last generated AND-node, T_{n+N}, matches or is the negation of the target truth table $T_\star$. Note that our environment is stateful, as its history influences future decisions. Furthermore, our environment poses an additional challenge due to its action space expanding quadratically with each node we add.

4 Model Architecture

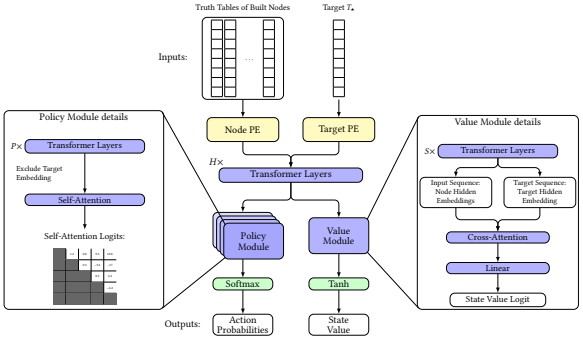


Figure 2: ShortCircuit model takes as inputs a target truth table $T_\star$ and the truth tables of the already built nodes. It first appends a type-dependent positional encoding before going through several transformer layers. Then, the model is split into two heads, respectively outputting a probability distribution over the next possible actions (policy module on the left) and a value reflecting the quality of the current inputs (value module on the right).

We propose an iterative approach to AIG construction, where we gradually build the circuit, starting with $\mathcal{G}_0$ and letting our model decide at each step which AND-node to add, aiming at realizing a given target truth table $T_\star$. To generate the next gate, the model takes the set of existing nodes as input, and it outputs a probability distribution over the set of AND-nodes that can be built by combining any pair of already existing nodes, taking edge types into account. Formally, let $|V_t|$ denote the number of nodes in the current state of the graph, then the action space is a $4 \times |V_t| \times |V_t|$ tensor, where each $|V_t| \times |V_t|$ slice corresponds to a connection type. Specifically, the cell (i, j) in a given slice indicates the probability of connecting node v_i with v_j, and the slice index $\epsilon \in \{1, 2, 3, 4\}$, corresponds to a combination of specific edge types: (v_i, v_j), $(\neg v_i, v_j)$, $(v_i, \neg v_j)$, or $(\neg v_i, \neg v_j)$. Therefore, we can sample a triplet (ϵ, i, j) following the distribution given by this $4 \times |V_t| \times |V_t|$ tensor and add the corresponding node to the graph. The process ends when the truth table of the node matches either the target one $T_\star$ or its full negation $\neg T_\star$, or reaches a maximum number of steps $N_{\max}$.

To effectively explore and prune the vast state space, our model comprises a policy and a value module to assess intermediate states and strategically get closer to the desired target. As

Table 2: Edges for each ϵ.

ϵ	Build v_k from (ϵ, i, j)
1	$v_i \wedge v_j$
2	$\neg v_i \wedge v_j$
3	$v_i \wedge \neg v_j$
4	$\neg v_i \wedge \neg v_j$

shown in Fig. 2, our architecture consists of a shared core embedding the truth tables applying position encodings, and H transformer encoder layers. Then, the hidden embeddings are passed as input to the 4 stacked policy modules and to the value module. The policy modules combined with a softmax produce a distribution over next actions, and the value module ending with tanh predicts an expected reward in $[-1, 1]$.

Positional Encoding (PE). In transformers, positional encodings (PE) help capture the sequential relations in the inputs. In our setting, the AIG structure is already implicitly reflected in the truth tables. Furthermore, self-attention should treat every node equally, as any two nodes may be combined to form a new node. However, to enable the model to distinguish between built nodes and the target truth table, we introduce two learnable positional encoders, referred to as "Node PE" (see Fig. 2).

Policy Module. Transformers are the state-of-the-art architecture to handle sequential data. These models particularly shine when trained to predict the next token in a sequence by outputting a fixed-size tensor representing a sampling probability over a token glossary. In our case though, the set of nodes that we can build grows at each step, as more pairs of nodes can be combined to produce the next node. Inspired by NLP tasks, for which attention scores among related tokens are high, we use attention to guide next node generation. Thus, we directly use the final self-attention map of four parallel policy modules to get the probability to connect any two nodes with specific edge types. Each policy module has P transformer encoder layers and outputs a final self-attention layer. In the last self-attention layer, we exclude the entry corresponding to the target node and returns the self-attention scores based on the existing AIG nodes embeddings. We aggregate the scores from the four policy modules and mask the ones associated to already built nodes and their negate versions. We finally apply a softmax to the remaining scores to produce a single probability distribution.

Value Module. The value module consists of S transformer encoder layers. It assess how favorable a state is and prevents expanding unpromising states. As the quality of a state not only depends on the nodes that are present in the graph at that stage, but also on the target truth table that should be realized, the value module also uses the two learnable type-based positional encoders introduced above. After performing the embedding, we compute the cross-attention between the graph nodes and the target truth table, which yields a new vector representation of the target. Finally, we feed this vector to a linear layer producing a single value, which should reflect the quality of the current state.

5 Training ShortCircuit

The sparse nature of the problem makes it practically impossible to discover functionally correct graphs when exploring the double exponential state space uniformly. This challenge necessitates a more effective training approach for our ShortCircuit. To address this, we propose a two-stage training regimen consisting of a supervised pre-training stage to initialize the policy module, followed by an AlphaZero-style fine-tuning phase to improve the policy module and train the value module. Although pre-training the policy module provides a good prior to predict the most useful next actions, simply following it does not guarantee that an AIG matching $T_\star$ will be constructed due to the inherent difficulty of the problem. This limitation highlights the importance of the fine-tuning stage, which aims to refine the policy module and leverage the value module for improved performance.

5.1 Pre-Training

Inspired from NLP, we pre-train our model on the next-action prediction task using single-output AIGs. As no large public corpus of (truth table, AIG) pairs exists, we curate our own dataset from open-source digital circuit collections to pre-train ShortCircuit to regenerate AIGs from their targets.

Data Extraction. To generate an AIG dataset with the desired input and output sizes, we utilize the EPFL benchmarks [2], which contain a collection of 20 real circuits realizing arithmetic and control functions. On average, the arithmetic and control circuits have 175 inputs and 137 outputs with 22520 AND-nodes. Since these circuits have more inputs than the AIGs we aim to generate, we extract subgraphs, or *cuts*, from them. A cut refers to a connected subset of nodes in the AIG that divides the graph into two disjoint parts. The root of a cut is the node to which all directed paths within the cut converge to and a *leaf node* is a node in the cut that have at least one fanin outside of the cut. By design, a cut forms a single-output AIG with a number of inputs corresponding to the number of leaves.

Data Preparation. Since our policy should predict the next action, i.e. the next node to add to a partial AIG, we need to convert the AIGs we load from our training dataset into a sequence of ground-truth actions. As different series of actions can lead to the same graph with N nodes, we first sort the nodes of the training AIG we load into a topological order $\{I_1, \ldots, I_n, \wedge_{n+1}, \ldots, \wedge_N\}$ (or $\{v_1, \ldots, v_N\}$ with node-type agnostic notation), where $\wedge_N$ is connected to the output O. We also convert the AIG nodes into truth tables, as described in section 2, and use the truth table of O as the target $T_\star$. From the topological sequence of nodes, we build the sequence of actions that our policy should learn to perform when its goal is to generate $T_\star$. As mentioned in section 4, creating node $v_k = (\neg)v_i \wedge (\neg)v_j$, with $1 \leq i < j < k$, corresponds to action $a_k = (\epsilon, i, j)$, whose first component $\epsilon \in \{1, 2, 3, 4\}$, indicates the types of the edges connecting v_k to its parents, as detailed in Table 2. This procedure leaves us with the sequence of actions $\mathcal{A} = \{a_{n+1}, a_{n+2}, ..., a_N\}$, starting with index $n + 1$ since the n primary inputs are given at the beginning of the AIG generation.

To efficiently generate target action distributions, we aggregate all the actions into a sparse 3-dimensional tensor $\mathbf{A} = (\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3, \mathbf{A}_4)$ where each element $\mathbf{A}_\epsilon$ is a $N \times N$ matrix representing the actions with connection type ϵ. The value of the entry (i, j) of $\mathbf{A}_\epsilon$ is set to 1 if (ϵ, i, j) belongs to $\mathcal{A}$ and to zero otherwise. Thus, if all the nodes up to v_k are already built, considering each submatrix $\mathbf{A}_{\epsilon,1:k,1:k}$ taking the first k rows and k columns of $\mathbf{A}_\epsilon$ allows to easily identify which nodes with connection ϵ we could build next. Taking the submatrices for all values of ϵ, we obtain the target action distribution by setting the entries corresponding to already performed actions at 0, and normalizing the resulting tensor. Fig. 3 illustrates this action tensor building procedure.

Data Augmentation. The first data augmentation we employ consists of using the same AIG for both targets $T_\star$ and $\neg T_\star$. This is valid because we can generate one target or the other by connecting the final node $\wedge_N$ with the output O using a regular or an inverter

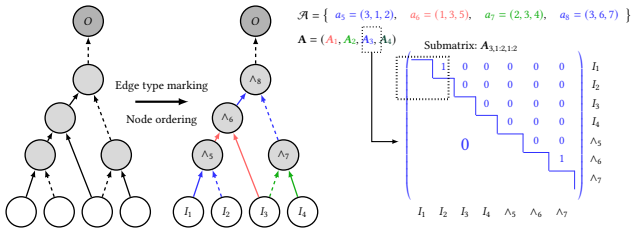


Figure 3: We start data pre-processing by sorting the AIG nodes in topological order. Then, we identify the action types $\epsilon \in \{1, 2, 3, 4\}$ based on the edges. Next, we build the sequence of actions $\mathcal{A}$ and generate the global action tensor $A = (A_1, A_2, A_3, A_4)$. We highlight the structure of A_3, which contains a 1 at entries $(1, 2)$ and $(6, 7)$ for the generation of $\wedge_5$ and $\wedge_8$ (actions a_5 and a_8).

edge. Our second data augmentation leverages the fact that any order of truth table rows is valid, provided that the same order is used for all the nodes. Since our ShortCircuit's inputs are truth tables, it is desirable for the model to be invariant to row permutations. Formally, the model should generate the same next-action prediction whether it receives the truth tables $T_1, \ldots, T_N$ and $T_\star$, where $T_i = (t_i^{(1)}, \ldots, t_i^{(2^n)}) \in \{0, 1\}^{2^n}$, or when it gets the permuted truth tables $\sigma(T_1), \ldots, \sigma(T_N), \sigma(T_\star)$ where σ is a permutation in $\mathbb{S}_{2^n}$ and $\sigma(T_i) = \left(t_i^{(\sigma(1))}, \ldots, t_i^{(\sigma(2^n))}\right)$. As structurally encoding this invariance into our policy architecture would be computationally too expensive, we apply random permutations to the inputs of our model during the training, which does not impact the other metadata introduced in the previous section.

Pre-Training Flow. With the prepared augmented data, we can proceed to train our policy module to match the ground-truth next-action distributions of our training set. For training loss, we experimented with KL divergence and cross-entropy, both of which measure the distance between two probability distributions. In practice, KL divergence loss yielded better results. Besides, the backbone of our model being a transformer, we implement a custom masking strategy during training. to maintain causality in the auto-regressive generation process. Since the primary inputs and the target truth table are available from the start, and as there is no causality for their existence, we allow full attention for their embeddings, and only apply a causal mask for the rest of the nodes.

5.2 Fine-Tuning

Fine-tuning aims to align the value and policy module to operate effectively together. Unlike the policy module, we cannot properly initialize the value module during pre-training as the generated dataset only contains successful examples, which would mislead the value module to consider that all states are "good". Skipping pre-training, though, would lead to a random exploration of the vast search space of truth tables, which would likely result in encountering only "bad" states, preventing the model from learning what a "good" state is. Therefore, the most viable option to train our value module is through experience, by performing searches with a pre-trained policy module. We utilize AlphaZero as the orchestration framework to refine the policy and value modules.

Fine-Tuning Flow. Generating trajectories for millions of truth tables is computationally challenging. Thus, to best exploit our resources, our fine-tuning regimen consists of data collection and model training processes. These data collectors generate trajectories and add their findings to a fixed length replay buffer. Under the hood, the data collectors store the metadata, including truth tables and discovered reward $Q(s)$, of the MCTS root node for each step in the trajectory in the replay buffer. Successful trajectories receive a reward of 1, while failed ones receive $-\min\left(h_d(T_N, T_\star), h_d(T_N, \neg T_\star)\right)$, where h_d is the normalized Hamming distance and T_N is the last generated truth table. The trainer process randomly samples data from the replay buffer and uses the truth tables as input for the model. Since the value module aims to predict the Q-value of a state, the goal is to minimize the mean squared error (MSE) between the predicted value and the retrieved $Q(s)$. The target distribution for the policy module is the normalized number of visitations $N(s,a)/\sum_a N(s,a)$. Similar to pre-training, we minimize the KL-divergence between the policy module output and the target probability distribution.

6 Experimental Evaluation

We introduce the implementation details such as model and search hyperparameters, datasets and baseline methods in section 6.1 and 6.2, and we compare the effectiveness of ShortCircuit against several baselines in 6.3. Finally, section 6.4 presents a study on the impact of number of simulations.

6.1 Experimental Setup

We train and evaluate ShortCircuit with 51.6 million parameters on 8-input truth tables randomly extracted from the EPFL benchmark, as we describe in section 5.1. Our model has $H = 4$ and $P = S = 3$ transformer blocks for the different parts with 16 heads and an intermediate embedding size of 4096. We specifically choose to test on these circuits, since they correspond to real-world Boolean functions that have more practical interest than uniformly random truth-tables. In total, we extract 1.8 million AIGs with an average number of AND-nodes of 10.08. We pre-train ShortCircuit with a batch size of 1024 for 250 epochs, and finetune the model until it converges.

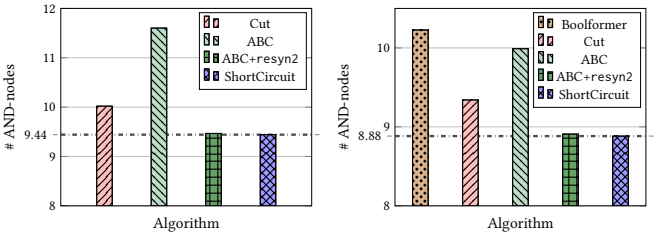
6.2 Baselines

We derive each truth table in our test set from the primary output of an extracted cut. Since those cuts are AIGs realizing those truth tables, we use them as a baseline, denoted as `Cut`. Additionally, we compare ShortCircuit against the state-of-the-art open-source logic synthesis tool `ABC`. This library applies a series of Boolean algebra transformations to generate an AIG from a truth table. Moreover, we leverage a popular logic optimization flow, `resyn2`, that applies multiple operators to optimize the AIGs by `ABC` and denote it as `ABC+resyn2`. Finally, we compare ShortCircuit against the Boolformer. This learned method produces an optimized Boolean expression given a truth table, and we convert this expression into an AIG without introducing any logic redundancy.

6.3 Generation Quality & Runtime Experiments

We evaluate ShortCircuit against our baselines on 500 truth tables associated with randomly sampled AIGs from the EPFL benchmarks. We allow ShortCircuit to attempt to generate a circuit with up to 30

AND-nodes. ShortCircuit performs 8 MCTS simulations and generates up to 20 AND-nodes in each simulation, before performing an action. The success rate of ShortCircuit on this test set is 98%.



(a) ShortCircuit vs. traditional methods

(b) ShortCircuit vs. traditional and learned methods

Figure 4: Average number of AND-nodes for the successfully generated AIGs across several baselines.

Fig. 4a compares the average size of the successfully generated circuits compared to `Cut`, `ABC`, and `ABC+resyn2`. ShortCircuit directly generates compact AIGs with 9.44 AND-nodes on average. These AIGs are significantly smaller than the ones from `Cut` and `ABC`, respectively achieving a size reduction of 5.77% and 18.62%, and marginally smaller AIGs than `ABC+resyn2` by 0.26%.

Fig. 4b compares the AIG sizes against the same baselines but also against Boolformer. Boolformer successfully generated 85% of the given truth tables, so we report results only on the truth tables successfully generated by both learned methods. ShortCircuit still maintains its good performance on this test subset while Boolformer produces less compact AIGs and frequently fails to synthesize complex circuits requiring more AND nodes. ShortCircuit's AIGs are 13.14% smaller.

The runtime of ShortCircuit for a single rollout with 30 steps is 0.01s. Due to limitations in our MCTS implementation, the runtime for 8 simulations per step is 1.8s, but an optimized version would be comparable by performing batched simulations. When we compare the greedy rollout against `ABC` and `ABC+resyn2`, which have a runtime of 0.31s and 0.32s, ShortCircuit achieves a respective speedup of 31× and 32×. Finally, Boolformer's runtime is 0.78s, making ShortCircuit 78× faster.

6.4 Impact of Number of Simulations

To better understand the role of MCTS simulations in the performance of ShortCircuit, we investigate their impact on the success rate, the circuit size, and the execution time on the same test set of 500 truth tables as in section 6.3. Fig. 5 illustrates how the success rate evolves as the number of MCTS simulations increases from 1 to 256. For clarity, we append an integer i next to our method's name (ShortCircuit[i]) to indicate the number of simulations.

When using only 1 simulation, ShortCircuit[1] performs a greedy search, where the policy selects the most likely action. This strategy yields a lower success rate of 92.2% albeit still good, but benefits from a very short generation time of 0.01s. On the other end, ShortCircuit[256] achieves a significantly higher success rate of 98.6%, albeit with a much longer running time of 106s. Increasing the number of simulations enables the model to explore a larger – but still limited – portion of the solution space, resulting in higher success rates and the discovery of more compact graphs. Fig. 6 highlights this trade-off by revealing a Pareto front, suggesting that we can adjust the number of MCTS simulations to achieve the desired balance between success rate, design quality, and running time. Moreover, Fig. 6 confirms that, with more engineering efforts, the runtime for different numbers of simulations would be much closer to ShortCircuit[1], since the runtime scales sublinearly with respect to the number of simulations as already visited states do not need to be recomputed and are retrieved from cache.

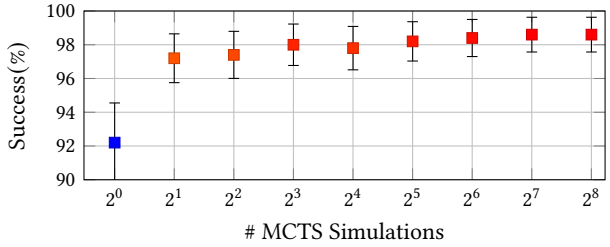


Figure 5: ShortCircuit's success rate vs. number of MCTS simulations per action.

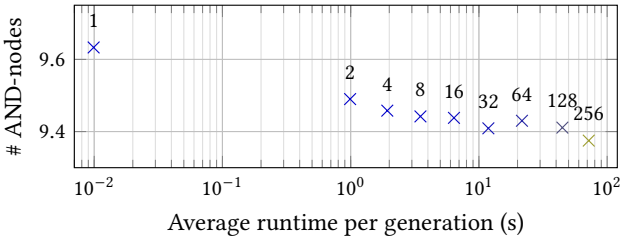


Figure 6: Average #AND-nodes per AIG vs. generation time for ShortCircuit with varying MCTS simulations marked on top of each data point.

7 Conclusion

We introduced ShortCircuit, a novel transformer-based architecture for generating AIGs from a target truth table. Our approach combines a structurally aware transformer model with an AlphaZero-inspired policy variant, enabling efficient navigation through the doubly exponential state space associated with truth tables. In our experiments, we demonstrated the effectiveness of ShortCircuit in producing high-quality AIGs that are significantly smaller than those generated by one of the state-of-the-art logic synthesis tools `ABC` and the trained Boolformer. Specifically, our method achieved a relative size reduction of 18.62%, and 15.13% respectively.

This work contributes to the expanding field of ML applications in chip design, showcasing the potential of deep learning to revitalize the field with new perspectives. We demonstrated that it is possible to generate high-quality AIGs from truth tables, paving the way for future research in this area. Future work will focus on extending ShortCircuit to handle AIGs with multiple outputs, integrating our approach with existing logic synthesis tools, and exploring its application in industrial settings. Finally, our goal is to enable the creation of efficient, scalable, and innovative computing systems, and we believe that ShortCircuit is an important step towards realizing this vision.

8 Acknowledgement

This work is partially supported by Hong Kong Research Grants Council (RGC) CRF-YCRG C6003-24Y.

References

[1] Alawieh, M. B., Li, W., Lin, Y., Singhal, L., Iyer, M. A., and Pan, D. Z. High-definition routing congestion prediction for large-scale fpgas. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)* (2020), pp. 26–31.

[2] Amarú, L., Gaillardon, P.-E., and De Micheli, G. The epfl combinational benchmark suite. In *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)* (2015), no. CONF.

[3] Amarú, L., Vuillod, P., Luo, J., and Olson, J. Logic optimization and synthesis: Trends and directions in industry. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017* (2017), pp. 1303–1305.

[4] Basak Chowdhury, A., Tan, B., Carey, R., Jain, T., Karri, R., and Garg, S. Bulls-eye: Active few-shot learning guided logic synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 42*, 8 (2023), 2580–2590.

[5] Belcak, P., and Wattenhofer, R. Neural combinatorial logic circuit synthesis from input-output examples. *CoRR abs/2210.16606* (2022).

[6] Darringer, J. A., Joyner, W. H., Berman, C. L., and Trevillyan, L. Logic synthesis through local transformations. *IBM Journal of Research and Development 25*, 4 (1981), 272–280.

[7] d'Ascoli, S., Bengio, S., Susskind, J. M., and Abbe, E. Boolformer: Symbolic regression of logic functions with transformers, 2024.

[8] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics* (2019).

[9] Dong, Z., Cao, W., Zhang, M., Tao, D., Chen, Y., and Zhang, X. Cktgnn: Circuit graph neural network for electronic design automation. *arXiv preprint arXiv:2308.16406* (2023).

[10] Feng, C., Lyu, W., Chen, Z., Ye, J., jie Yuan, M., and Hao, J. Batch sequential black-box optimization with embedding alignment cells for logic synthesis. *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)* (2022), 1–9.

[11] Grosnit, A., Malherbe, C., Tutunov, R., Wan, X., Wang, J., and Ammar, H. B. Boils: Bayesian optimisation for logic synthesis. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (2022), IEEE, pp. 1193–1196.

[12] Gubbi, K. I., Beheshti-Shirazi, S. A., Sheaves, T., Salehi, S., PD, S. M., Rafatirad, S., Sasan, A., and Homayoun, H. Survey of machine learning for electronic design automation. In *Proceedings of the Great Lakes Symposium on VLSI 2022* (New York, NY, USA, 2022), GLSVLSI '22, Association for Computing Machinery, p. 513–518.

[13] Haaswijk, W., Collins, E., Seguin, B., Soeken, M., Kaplan, F., Süsstrunk, S., and De Micheli, G. Deep learning for logic optimization algorithms. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)* (2018), pp. 1–4.

[14] Hillier, A., Vũ, N. N., Mankowitz, D. J., Calandriello, D., Leurent, E., Rotival, G., Lobov, I., Mahajan, K., Gelmi, M., and Antropova, N. Learning to design efficient logic circuits, 2023. NANDA Workshop 2023.

[15] Hosny, A., Hashemi, S., Shalan, M., and Reda, S. DRiLLS: Deep Reinforcement Learning for Logic Synthesis. *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)* (Sept. 2020), 581–586.

[16] Huang, G., Hu, J., He, Y., Liu, J., Ma, M., Shen, Z., Wu, J., Xu, Y., Zhang, H., Zhong, K., Ning, X., Ma, Y., Yang, H., Yu, B., Yang, H., and Wang, Y. Machine learning for electronic design automation: A survey. *ACM Trans. Design Autom. Electr. Syst. 26* (2021), 40:1–40:46.

[17] Jumper, J. M., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A., Ballard, A., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., Back, T., Petersen, S., Reiman, D., Clancy, E., Zielinski, M., Steinegger, M., Pacholska, M., Berghammer, T., Bodenstein, S., Silver, D., Vinyals, O., Senior, A. W., Kavukcuoglu, K., Kohli, P., and Hassabis, D. Highly accurate protein structure prediction with alphafold. *Nature 596* (2021), 583 – 589.

[18] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *ArXiv abs/2001.08361* (2020).

[19] Karnaugh, M. The map method for synthesis of combinational logic circuits. *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics 72* (1953), 593–599.

[20] Li, M., Shitole, V., Chien, E., Man, C., Wang, Z., Srinivas, Zhang, Y., Krishna, T., and Li, P. LayerDAG: A layerwise autoregressive diffusion model of directed acyclic graphs for system. In *Machine Learning for Computer Architecture and Systems 2024* (2024).

[21] Li, X., Li, X., Chen, L., Zhang, X., Yuan, M., and Wang, J. Circuit transformer: A transformer that preserves logical equivalence. In *The Thirteenth International Conference on Learning Representations* (2025).

[22] McCluskey Jr., E. J. Minimization of boolean functions. *Bell System Technical Journal 35*, 6 (1956), 1417–1444.

[23] Mishchenko, A., Brayton, R., Jang, S., and Kravets, V. Delay optimization using sop balancing. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (2011), pp. 375–382.

[24] Mishchenko, A., and Brayton, R. K. Scalable logic synthesis using a simple circuit structure.

[25] Mishchenko, A., Chatterjee, S., and Brayton, R. Dag-aware aig rewriting a fresh look at combinational logic synthesis. In *Proceedings of the 43rd Annual Design Automation Conference* (New York, NY, USA, 2006), DAC '06, Association for Computing Machinery, p. 532–535.

[26] Mishchenko, A., et al. Abc: A system for sequential synthesis and verification. *URL http://www. eecs. berkeley. edu/alanmi/abc 17* (2007).

[27] Peruvemba, Y. V., Rai, S., Ahuja, K., and Kumar, A. Rl-guided runtime-constrained heuristic exploration for logic synthesis. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)* (2021), IEEE Press, p. 1–9.

[28] Qian, Y., Zhou, X., Zhou, H., and Wang, L. An efficient reinforcement learning based framework for exploring logic synthesis. *ACM Trans. Des. Autom. Electron. Syst. 29*, 2 (jan 2024).

[29] Quine, W. V. The problem of simplifying truth functions. *The American Mathematical Monthly 59*, 8 (1952), 521–531.

[30] Quine, W. V. A way to simplify truth functions. *The American Mathematical Monthly 62*, 9 (1955), 627–631.

[31] Riener, H., Testa, E., Haaswijk, W., Mishchenko, A., Amarù, L., Micheli, G. D., and Soeken, M. Scalable generic logic synthesis: One approach to rule them all. In *2019 56th ACM/IEEE Design Automation Conference (DAC)* (2019), pp. 1–6.

[32] Roy, R., Raiman, J., Kant, N., Elkin, I., Kirby, R., Siu, M., Oberman, S., Godil, S., and Catanzaro, B. Prefixrl: Optimization of parallel prefix circuits using deep reinforcement learning. In *2021 58th ACM/IEEE Design Automation Conference (DAC)* (2021), pp. 853–858.

[33] Rudell, R., and Sangiovanni-Vincentelli, A. Multiple-valued minimization for pla optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 6*, 5 (1987), 727–750.

[34] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T. P., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. Mastering the game of go with deep neural networks and tree search. *Nat. 529*, 7587 (2016), 484–489.

[35] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. P., Simonyan, K., and Hassabis, D. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR abs/1712.01815* (2017).

[36] Tsaras, D., Li, X., Chen, L., Xie, Z., and Yuan, M. Elf: Efficient logic synthesis by pruning redundancy in refactoring. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)* (San Francisco, CA, USA, 2025).

[37] Tu, K., Tang, X., Yu, C., Josipović, L., and Chu, Z. *Logic Synthesis.* Springer Nature Singapore, Singapore, 2024, pp. 135–164.

[38] Ward, S. I., Kim, M.-C., Viswanathan, N., Li, Z., Alpert, C. J., Swartzlander, E. E., and Pan, D. Z. Keep it straight: teaching placement how to better handle designs with datapaths. In *ACM International Symposium on Physical Design* (2012).

[39] Wolf, C., Glaser, J., and Kepler, J. Yosys-a free verilog synthesis suite.

[40] Wu, N., Lee, J., Xie, Y., and Hao, C. Lostin: Logic optimization via spatio-temporal information with hybrid graph models. In *2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)* (2022), pp. 11–18.

[41] Yu, C., Xiao, H., and De Micheli, G. Developing synthesis flows without human knowledge. In *Proceedings of the 55th Annual Design Automation Conference* (New York, NY, USA, 2018), DAC '18, Association for Computing Machinery.

[42] Yu, C., and Zhou, W. Decision making in synthesis cross technologies using lstms and transfer learning. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)* (2020), pp. 55–60.

[43] Zhou, G., and Anderson, J. H. Area-driven fpga logic synthesis using reinforcement learning. In *2023 28th Asia and South Pacific Design Automation Conference (ASP-DAC)* (2023), pp. 159–165.

[44] Zhu, K., Liu, M., Chen, H., Zhao, Z., and Pan, D. Z. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)* (2020), pp. 145–150.

[45] Zimmer, M., Feng, X., Glanois, C., JIANG, Z., Zhang, J., Weng, P., Li, D., HAO, J., and Liu, W. Differentiable logic machines. *Transactions on Machine Learning Research* (2023).